INTERVIEW PREPARATION DOSSIER

AI PR Review Agent

Architecture, end-to-end request trace, and a local runbook for the autonomous GitHub pull-request review system.

Repository D:\users\AI-ML\powerful

Prepared 25 October 2026

Basis README, dependency manifests, full git history (19 commits), source code

3	8	4	19
SERVICES	GRAPH NODES	EXTERNAL SYSTEMS	COMMITS

CONTENTS

01 **The project in three sentences**
Problem, intended user, output — with evidence and commit history

02 **End-to-end trace of a pull request**
Trigger, input gathering, preparation, agent logic, output posting

03 **Worked example: PR #482, three files**
Risk math, chunking, the prompt, the LLM response, the GitHub review

04 **Local runbook**
Docker and bare-metal setup, environment variables, sample PR, logs

05 **Interview gotchas**
Code/README mismatches, failure modes, what not to overclaim

A **Appendix A — File map of the review path**
Every hop from webhook to GitHub review

B

Appendix B — Full commit history

All 19 commits, grouped by phase

SECTION 01

The project in three sentences

Short answers first, then the exact files, functions, and commits that support each one.

---

1 — WHAT PROBLEM DOES IT SOLVE?

It removes manual first-pass PR review: when a pull request is opened or updated, it reads the diff, has an LLM flag bugs, security issues, and bad patterns, posts a native GitHub review on the exact changed lines, and learns from the team's approve/dismiss decisions so later reviews follow that team's conventions.

Evidence. `README.md:11` describes an "autonomous, production-grade Pull Request Review engine"; `README.md:85` promises native reviews with `path`, `line`, `side` annotations. In code, `agent/app/services/github.py:93` `post_pull_request_review()` calls `pr.create_review(commit, body, event, comments)` at line 122, with an issue-comment fallback at line 137. The learning loop is `agent/app/services/memory.py` (`store_decision()`, `search_memory()`) against `MemoryEntry` with a `vector(384)` column (`api/prisma/schema.prisma:94–107`).

2 — WHO IS THE INTENDED USER?

Engineering teams on GitHub, organised as an organization — developers and tech leads who install a GitHub App on the org's repositories and then use a dashboard to watch reviews and give feedback.

Evidence. `Organization` is the tenancy root in `api/prisma/schema.prisma:12`, with `installationId`, `plan`, `monthlyReviewCount`, and relations to users, repos, reviews, digests, and memory entries; every `User` carries an `orgId` (line 40). `README.md:87` states "complete tenant isolation across organizations" and `README.md:129` tells you to create the App in organization settings. Commit `cac6b7d` ("look up org login from GitHub API in installation callback", 20 Jul 2026) is a fix that only matters when the organization is the unit of installation.

3 — WHAT OUTPUT DOES IT PRODUCE?

Three deliverables: a native GitHub PR review (approve, comment, or request changes, with inline annotations), a live Next.js dashboard with SSE pipeline progress, review history and learned-convention memory, and a weekly digest plus monthly quota reset driven by cron.

Evidence. Review output: `agent/app/graph/review/nodes/post_comments.py:11–30` selects `APPROVE` / `REQUEST_CHANGES` / `COMMENT`. Dashboard and streaming: `api/src/routes/internal.ts:93` `handlePipelineStream()`, exposed at `api/src/app.ts:106`, with pages under `frontend/src/app/dashboard/`. Digest and quota: `api/src/lib/cron.ts:34` `startCronJobs()` schedules Monday 09:00 (`'0 9 * * 1'`, line 36) and the first of the month (`'0 0 1 * *'`, line 45) via `digestQueue` (`api/src/queues/digestQueue.ts`).

1.1 Commit history at a glance

The repository contains **19 commits total**, so "the first 30 commits" is the entire history. Three phases are visible:

Phase	Commits	What changed
Initial version	`4beb79f` 19 Jul 2026	Entire three-service system in one commit: `agent/` (FastAPI + LangGraph, two graphs), `api/` (Express + Prisma + BullMQ), `frontend/` (Next.js), Docker, initial migration. The 78-line README already pitches RAG-based learning.
Deployment fixes	`afc4c95`, `e51d303`, `eb69b8e`, `07f4b75`, `800e280`, `01a36b9`, `cac6b7d` 19–20 Jul 2026	Submodule path, header TypeScript errors, `moduleResolution: Node16`, Render build errors, cross-site cookies for Vercel, organization lookup during installation. No new capability.
Redesign	`27d4457`, `5116b5b`, `17c7358` 12 Sep 2026	"fixes all inconsistency", "redesign frontend", "latest". Diffs are confined to `frontend/src/app/dashboard/*` and `frontend/src/components/*`.
Agent upgrade	`5c39865` 19 Sep 2026	The substantive post-launch change: adds `score_risk.py` (137 lines) and `fetch_config.py` (76 lines), edits `llm_review.py`, `api/src/app.ts`, and `schema.prisma`. Risk scoring and per-repo rules enter the pipeline.
Homepage polish	`1d3bc06`, `478dd74`, `5a43016`, `6e346e8`, `b5f5ab2`, `fe9b4e6`, `9d80155` 19–20 Sep 2026	Hero page iterations: "hompage design", "hompage interactive", "hompage design part 2", "fix hero page". Marketing UI, not the review path.

1.2 README versus code — say these carefully

DO NOT REPEAT THE README'S EMBEDDING CLAIM

`README.md:86` and `:120` claim `all-MiniLM-L6-v2`. The code actually loads **`BAAI/bge-small-en-v1.5`** in `agent/app/services/embeddings.py:4`, via `fastembed`. Both produce 384-dimensional vectors, which is why the schema column (`vector(384)`) is still correct. If asked which model, say bge-small and note the README is stale.

OTHER SMALL MISMATCHES

Memory scoping. The README says memories are scoped by `orgId` and `repoId` (`README.md:121`), but `search_memory()` filters only on `repoId` (`agent/app/services/memory.py:90-92`). The `MemoryEntry` row does store `orgId`, and writes are given it, but retrieval does not use it.

Frontend version. The README badge says Next.js 14 (`README.md:3`); `frontend/package.json:19` pins Next `16.2.10`.

No tests, CI config, or LICENSE file are present in the tree, despite `README.md:9` linking to a license.

SECTION 02

End-to-end trace of a pull request

What actually runs, in order, with the file and function responsible at each hop and the data crossing the boundary.

2.1 Entry points

There is no single `main()`. The PR flow enters through an HTTP route; each service has its own process entry point.

Service	Process entry	PR-flow entry
Node API	`api/src/app.ts` → `node dist/app.js` (`api/Dockerfile:35`)	`POST /webhooks/github` → handler in `api/src/routes/webhooks.ts`, mounted at `api/src/app.ts:89`
Python agent	`agent/app/main.py` → uvicorn `app.main:app`	`POST /review` → `review_pr()` in `agent/app/api/routes.py:74`
Next.js	`frontend/package.json` scripts `dev` / `start` on port 3001	Dashboard only — not in the review path

> **WHY THE RAW BODY MATTERS**
>
> `api/src/app.ts:81` registers `express.raw({ type: 'application/json' })` for `/webhooks` **before** `express.json()` at line 82. HMAC verification needs the exact bytes GitHub signed; a parsed-and-reserialized body would not match.

2.2 The path at a glance

1. **GitHub webhook** GITHUB
   `pull_request` opened or synchronize
2. **HMAC verification** NODE API
   `verifyWebhookSignature()` — raw body + `x-hub-signature-256`
3. **Guards + review row** NODE API
   dedup → org → quota → repo → `PRReview(status: pending)`
4. **Queue** REDIS / BULLMQ
   `review-queue`, 3 attempts, exponential backoff
5. **Worker + context** NODE API
   compute `pastDismissalRate`, call `triggerReview()`
6. **Agent graph** PYTHON
   8 nodes: fetch → config → chunk → memory → risk → LLM → post → notify

7 **GitHub review** GITHUB
`create_review()` with inline comments, or issue-comment fallback

8 **Persistence + telemetry** POSTGRES
comments, risk score, usage event, monthly counter

9 **Live dashboard** REDIS → SSE
`pipeline:<job_id>` pub/sub + 1-hour history replay

2.3 Step by step

STEP 0 Trigger

`GitHub → POST /webhooks/github` · `api/src/routes/webhooks.ts`

A developer opens a PR or pushes new commits to it. GitHub delivers a `pull_request` event (the App is subscribed to it, `README.md:148`) with header `x-github-event: pull_request`.

In: GitHub JSON payload (repository, pull_request, installation, organization, sender). **Out:** HTTP request to the Node API.

STEP 1 Verify the signature

`api/src/middleware/webhook.ts → verifyWebhookSignature()`

Reads `GITHUB_WEBHOOK_SECRET`, computes `sha256=HMAC(rawBody)`, and compares it against `x-hub-signature-256` with `crypto.timingSafeEqual` (length checked first). Missing secret → 500; missing, malformed, or wrong signature → 401.

In: raw `Buffer` + signature header. **Out:** `next()` or HTTP 401.

STEP 2 Route the event, apply guards, create the review row

`api/src/routes/webhooks.ts → router.post('/github')`

Parses JSON, then acts only on `action === 'opened' || 'synchronize'` (line 117). Guards run in this order:

- **Idempotency** — Redis `SET webhook:dedup:<repoId>:<pr>:<action>:<sha> NX EX 60` (line 130). Duplicate → 200 `duplicate_webhook`.
- **Org upsert** by `repository.owner.id` (lines 140–144), then skip if `org.isInstalled === false` (line 147).
- **Monthly quota** via `checkAndResetOrgQuota()`; limits `free: 50`, `pro: 500`, `enterprise: 10000` (lines 8–12).
- **Repo upsert** by `repository.id` (lines 170–180); skip if `repo.enabled === false` (line 190).
- **Skip synchronize** when `org.triggerOnSync === false` (line 183).

Then creates `PRReview { status: 'pending' }` (line 197) and enqueues a BullMQ job whose `job_id` is that row's `id` (line 207).

In: validated webhook JSON. **Out:** Organization, Repo, PRReview(pending) rows + one queued job; HTTP 200 (also on internal failure, deliberately, to stop GitHub retries).

STEP 3 Queue and worker pickup

`api/src/queues/reviewQueue.ts → api/src/workers/reviewWorker.ts`

The queue is `review-queue` with `attempts: 3` and exponential backoff from 5 s (`reviewQueue.ts:12–23`). The worker is constructed at import time (`reviewWorker.ts:8`, imported by `app.ts:16`) with `concurrency: 3` and a 5-minute lock. It loads the `PRReview` plus org; if the row is missing it skips without retry.

It then counts `FeedbackAction` rows for the org and computes `pastDismissalRate = dismissed / total`, defaulting to `0.2` when there is no history (lines 26–32).

In: job data `{ job_id, repo, pr_number, installation_id, head_sha }`. **Out:** HTTP `POST ${AGENT_URL}/review` via `triggerReview()` (`api/src/services/agent.ts:6`), 300 s timeout, with `x-internal-secret`.

STEP 4 Agent authentication and graph start

`agent/app/api/routes.py → verify_internal_secret() · review_pr()`

Every agent route depends on `verify_internal_secret()` (line 12, applied at line 37). Header `x-internal-secret` must equal `INTERNAL_SERVICE_KEY`; an unset server key returns 500, a wrong one 403. Startup validation (`agent/app/core/config.py:24`) hard-fails if `INTERNAL_SERVICE_KEY`, `GROQ_API_KEY`, or `DATABASE_URL` is missing.

`review_pr()` builds the initial state and calls `review_graph.ainvoke({...})` (line 78).

In: `{ job_id, repo, pr_number, installation_id, confidence_threshold, past_dismissal_rate, head_sha }`. **Out:** a completed `ReviewState` serialized into JSON for the Node API.

STEP 5 Fetch the diff

`agent/app/graph/review/nodes/fetch_pr.py → fetch_pr()` · `agent/app/services/github.py → get_pr_details(), get_github_client()`

The agent exchanges its installation id for an installation token: it calls `POST ${NODE_API_URL}/internal/installation-token` (`agent/app/services/github.py:12–21`), which lands on `api/src/routes/internal.ts:9` → `getInstallationOctokit()` in `api/src/lib/octokit.ts` . That function caches in Redis under `installation_token:<id>` for 55 minutes (lines 31–43, 62); on a miss it signs an RS256 app JWT (`generateJWT()` , line 8) and calls GitHub's `/app/installations/<id>/access_tokens` (lines 46–56).

With the token, `get_pr_details()` iterates `pr.get_files()` and keeps `{filename, patch, additions, deletions, status}` ; **files without a patch are dropped** (binary or too large), `github.py:36–44` .

In: repo full name, PR number, installation id. **Out:** `changed_files[]` + `head_sha` .

STEP 6 Load repository custom rules

`agent/app/graph/review/nodes/fetch_config.py → fetch_config()`

Looks for `.powerful.yml` , then `.powerful.yaml` , at the PR head SHA (line 30). Base64-decodes the contents, runs `yaml.safe_load` , and accepts either `{ rules: [...] }` or a bare YAML list (lines 36–41). A 404 is expected and ignored; malformed YAML is logged and the review proceeds with no rules.

In: repo + `head_sha` . **Out:** `custom_rules: string[]` (possibly empty).

STEP 7 Chunk the diff for the LLM

`agent/app/graph/review/nodes/chunk.py → chunk_changes()`

Per file, walk patch lines: `@@` starts a new chunk and parses the new-file start line; lines starting with `+` , `-` , or a space are kept; `-` lines do not advance the new-file line counter (line 37); a chunk is flushed every **30 lines** (line 39).

In: `changed_files[]` . **Out:** `chunks[] = { filename, content, start_line, end_line }` .

STEP 8 Retrieve team memory (RAG)

`agent/app/graph/review/nodes/search.py → search_memory_for_chunks()` · `agent/app/services/memory.py → search_memory()`

Each chunk is embedded with `embed()` (`agent/app/services/embeddings.py`, `BAAI/bge-small-en-v1.5`, 384 dimensions). SQL orders by cosine distance and returns the top 3 per chunk:

```
SELECT "content", "outcome", "prNumber",
       1 - ("embedding" <=> $1::vector) AS similarity
FROM "MemoryEntry"
WHERE "repoId" IN (SELECT "id" FROM "Repo" WHERE "id" = $2 OR "fullName" = $2 OR "name" = $2)
ORDER BY "embedding" <=> $1::vector
LIMIT $3
```

Only hits with `similarity > 0.4` survive (`search.py:16`).

In: chunk text. **Out:** `chunks_with_memory[]`, each chunk plus up to 3 past decisions.

STEP 9 Score PR risk

`agent/app/graph/review/nodes/score_risk.py → score_risk()`

Four weighted buckets, clamped to 5–100 (lines 79–80): diff size 5/12/20/26/30 points for ≤50/≤200/≤500/≤1000/>1000 lines; file count 4/9/15/20 for 1–2/3–5/6–10/>10 files; criticality from **filename regexes** (one hit per category, capped at 35) — auth/session/security 18, payments 15, infrastructure/schema 12, core logic 8; team dismissal friction `min(15, max(2, round(rate × 25)))`.

It immediately posts a commit status via `post_commit_status()` (`agent/app/services/github.py:54`) with context `Powerful AI / PR Risk`: `success` below 30, `pending` at 30–70, `failure` above 70.

In: changed files + dismissal rate. **Out:** `risk_score`, `risk_breakdown`, and a GitHub commit status.

STEP 10 LLM review

`agent/app/graph/review/nodes/llm_review.py → llm_review()` · `agent/app/services/llm.py`

One Groq call **per chunk** (line 28). Model: `llama-3.3-70b-versatile`, temperature `0.1`. System message: "You are a precise code reviewer. Output only valid JSON arrays.", plus any custom rules. The human message contains the filename, line range, raw diff, optional rules, optional memory precedents, and a strict JSON output schema.

Responses are stripped of Markdown fences, parsed with `json.loads`, and every issue with `confidence < confidence_threshold` is dropped (lines 80–82). A malformed chunk is logged and skipped — the run continues.

In: chunks + memory + rules + threshold. **Out:** `comments: ReviewComment[]`, `llm_calls`, prompt/completion/total tokens.

STEP 11 Post the GitHub review

`agent/app/graph/review/nodes/post_comments.py → post_comments()` · `agent/app/services/github.py → post_pull_request_review()`

Event selection (`post_comments.py:11–30`): no comments → `APPROVE` with "✅ PR Review Agent — No issues found. Looks good!"; any `error` severity → `REQUEST_CHANGES` ; otherwise `COMMENT` .

Builds a Markdown body grouped by file with severity icons (🔴 / ⚠️ / 💡), line number, comment, optional past-PR reference, and confidence (lines 32–46). Inline comments use `{ path, line, body }` (lines 48–57). The service calls `pr.create_review(commit=repo.get_commit(pr.head.sha), body=..., event=..., comments=...)` with `side: "RIGHT"` ; any exception falls back to a single `create_issue_comment(body)` , **losing inline annotations** (`github.py:109–138`).

In: comments + PR identity. **Out:** `posted_urls` , the GitHub review URL.

STEP 12 Notify, persist, and meter

`agent/app/graph/review/nodes/notify.py → notify_complete()` · `api/src/workers/reviewWorker.ts`

The agent posts `/internal/review-complete` with `{ job_id, comments_count, comment_url, status }` (line 12–22); `internal.ts:26` updates the `PRReview` row and emits a `done` event. The worker then writes the detailed record: `reviewComment.createMany()` , `PRReview` completion fields, `Repo.customRulesCount` , a second commit status with a dashboard link, a `UsageEvent` , and `Organization.monthlyReviewCount += 1` (lines 44–129).

On failure, `worker.on('failed')` marks the review `failed` (lines 137–147); BullMQ retries up to 3 times.

In: agent result. **Out:** completed DB rows, commit status, usage event, incremented quota counter.

STEP 13 Live progress, throughout

`agent/app/services/progress.py → report_progress()` · `api/src/lib/pipelineEvents.ts → emitPipelineEvent()` · `api/src/routes/internal.ts → handlePipelineStream()`

Every node calls `report_progress(job_id, node, status, message, meta)` , which POSTs to `/internal/pipeline-update` (`internal.ts:63`). `emitPipelineEvent()` publishes to `pipeline:<job_id>` **and** appends to `pipeline_history:<job_id>` with a 1-hour TTL. The SSE endpoint `GET /api/pipeline/stream/:job_id` replays history, subscribes to the channel, and heartbeats every 15 s.

In: node name, status, message, metadata. **Out:** SSE `data:` frames to the dashboard.

2.4 The state object the graph passes around

`agent/app/graph/review/state.py` defines `ReviewState` (a `TypedDict` , all keys optional) and a `ReviewComment` dataclass. Every node returns `{**state, ...new_keys}` , so the dict grows as it moves right.

Key	Produced by	Meaning
`job_id`, `repo`, `pr_number`, `installation_id`	Agent route	Work identity and credentials
`confidence_threshold`, `past_dismissal_rate`, `head_sha`	Agent route (from worker)	Org sensitivity, feedback-derived friction, commit anchor
`changed_files`, `head_sha`	`fetch_pr`	Filtered diff files with patches
`custom_rules`	`fetch_config`	Rules from `.powerful.yml`
`chunks`	`chunk_changes`	≤30-line slices with line ranges
`chunks_with_memory`	`search_memory`	Chunks plus matching past decisions
`risk_score`, `risk_breakdown`	`score_risk`	0–100 and the four components
`comments`	`llm_review`	Filtered `ReviewComment` objects
`posted_urls`	`post_comments`	GitHub review URL(s)
`llm_calls`, `prompt_tokens`, `completion_tokens`, `total_tokens`	`llm_review`	Telemetry written to `UsageEvent`

2.5 Decision points

Condition	Outcome	Where
Duplicate webhook (same repo/PR/action/SHA within 60 s)	200, ignored	`webhooks.ts:129–135`
Org uninstalled	200, ignored	`webhooks.ts:147–151`
Monthly quota reached	200, ignored	`webhooks.ts:153–167`
`synchronize` but `triggerOnSync=false`	200, ignored	`webhooks.ts:183–187`
Repo `enabled=false`	200, ignored	`webhooks.ts:190–194`
Zero issues survive the threshold	Review event `APPROVE`	`post_comments.py:11–21`
Any issue with `severity=error`	Review event `REQUEST_CHANGES`	`post_comments.py:29–30`
Warnings/suggestions only	Review event `COMMENT`	`post_comments.py:30`
`create_review` throws (e.g. invalid line)	Fallback to one issue comment	`github.py:135–138`

Condition	Outcome	Where
Agent call throws or times out	Retry ×3, then `status=failed`	`reviewQueue.ts:14–22`, `reviewWorker.ts:137–147`

SECTION 03

Worked example: PR #482, three files

A single concrete run, end to end. Control flow, prompt template, score formula, and output formatting are taken from the code; the specific code, issue text, and numbers are a constructed illustration.

READ THIS BEFORE QUOTING THE EXAMPLE

The pull request, file contents, LLM JSON, and comment wording below are **invented** to make the mechanics concrete. What is real: the graph order (`agent/app/graph/review/agent.py:24–32`), the chunking rule (`chunk.py`), the risk weights (`score_risk.py`), the prompt template (`llm_review.py:38–61`), and the Markdown format (`post_comments.py:32–57`).

3.1 Setup

REPOSITORY
acme/shop

PULL REQUEST
#482 — "Add remember-me sessions and coupon validation"

INSTALLATION
12345678

Org sensitivity is the schema default `0.5` (`schema.prisma:20`); the worker computes `pastDismissalRate = 0.20` (4 dismissals out of 20 feedback actions); there is no `.powerful.yml` .

File	Added	Removed	Hunks	Changed lines
`api/src/routes/auth.ts`	+18	−4	1	22
`api/src/services/billing.ts`	+41	−12	2	53
`web/src/components/Button.tsx`	+8	−3	1	11
Total	**+67**	**−19**	**4**	**86**

3.2 What the system does with it

#	Step	Result for this PR
1	Webhook	`prNumber=482` , `repo="acme/shop"` , `installationId=12345678` , `headSha="9f3c2ab…"` , action `opened` .
2	Guards	Fresh dedup key; org installed; 12 of 50 free-plan reviews used; repo enabled. Pass.
3	Row + job	`PRReview clxr482` created with `status=pending` ; job `review-pr` queued.
4	Worker context	`confidence_threshold=0.5` , `past_dismissal_rate=0.20` .
5	fetch_pr	3 changed files (all carry patches); `head_sha` confirmed.
6	fetch_config	Both `.powerful.yml` and `.powerful.yaml` return 404 → `custom_rules=[]` .

#	Step	Result for this PR
7	chunk_changes	**4 chunks** (table below).
8	search_memory	4 queries × 3 results; **one** hit above 0.4: billing chunk 1, PR #214, similarity 0.72.
9	score_risk	**59/100 · Moderate Risk · pending** (table below).
10	llm_review	4 Groq calls; 2 issues survive the 0.5 threshold.
11	post_comments	No `error` severities → event `COMMENT`; 2 inline comments.
12	notify + persist	Row completed, 2 `ReviewComment` rows, usage event, quota 12 → 13.
13	SSE	9 events streamed: 8 node transitions + `done`.

3.3 Risk score math

Bucket	Calculation	Points	Reason string produced
Diff size	86 lines → 51–200 band	12	"Moderate diff (51–200 lines)"
File count	3 files → 3–5 band	9	"Moderate scope (3–5 files)"
Criticality	`auth.ts` → Security 18; `billing.ts` → Payments 15; `Button.tsx` → none (cap 35)	33	Two matched categories, one per filename
Dismissal friction	`round(0.20 × 25) = 5`, clamped 2–15	5	"Team dismissal history: 20%"
Raw	12 + 9 + 33 + 5	**59**	Clamped to 5–100 → **59**
Band	30–70 → Moderate, commit state `pending`	**59**	"Risk Score: 59/100 (Moderate Risk)"

3.4 Chunking

Chunk	File	Lines	Length	Why it ends there
1	`api/src/routes/auth.ts`	35–56	22	End of file/hunk, under the 30-line cap
2	`api/src/services/billing.ts`	80–109	30	Flushed exactly at the 30-line cap
3	`api/src/services/billing.ts`	120–142	23	Second hunk, under the cap
4	`web/src/components/Button.tsx`	12–22	11	Single small hunk

3.5 The memory hit

RETRIEVED FOR BILLING CHUNK 1 — SIMILARITY 0.72

PR #214 · outcome dismissed · file `api/src/services/billing.ts`

"Dismissed/Rejected pattern on api/src/services/billing.ts: Don't add retry loops around Stripe calls; use the shared idempotency wrapper."

Stored by `store_decision()` when a developer dismissed the earlier comment (`memory.py:5–67`); surfaced because 0.72 > 0.4 (`search.py:16`).

3.6 The prompt sent to the LLM

SYSTEM MESSAGE

```
You are a precise code reviewer. Output only valid JSON arrays.
```

HUMAN MESSAGE — BILLING CHUNK 1 (ILLUSTRATIVE VALUES INSIDE THE REAL TEMPLATE)

```
You are reviewing a code diff for the file: api/src/services/billing.ts
Lines 80 to 109.

Code change:
@@ -80,7 +80,33 @@ export async function applyCoupon(...)
-  const charge = await stripe.charges.create({ amount, currency })
+  let charge
+  for (let attempt = 0; attempt < 3; attempt++) {
+    try {
+      charge = await stripe.charges.create({ amount, currency })
+      break
+    } catch (err) {
+      if (attempt === 2) throw err
+      await sleep(500 * 2 ** attempt)
+    }
+  }
+  if (!charge) throw new Error('charge failed')
+  ...


Your team's past decisions on similar code:

- PR #214 | outcome: dismissed | similarity: 0.72
  Dismissed/Rejected pattern on api/src/services/billing.ts: Don't add retry loops
  around Stripe calls; use the shared idempotency wrapper.


Should any part of this be flagged for review?
Verify that code changes adhere to standard engineering best practices AND all
repository custom rules listed above.

If YES — respond with a JSON array of issues found.
If NO issues — respond with an empty array [].

Respond ONLY with a JSON array, no markdown:
[
  {
    "line": <line number where the issue is>,
    "severity": "error|warning|suggestion",
    "comment": "specific actionable comment, reference past PR or custom rule if relevant",
    "confidence": <0.0 to 1.0>,
    "past_pr_number": <PR number if referencing past decision, or null>
  }
]
```

3.7 What the model returns

One response per chunk. Two survive the `0.5` confidence filter; the other two are empty arrays.

```
// chunk 1 — auth.ts
[{"line":42,"severity":"warning",
  "comment":"Session cookie is set without httpOnly/sameSite; an XSS on any page can steal the
remember-me token. Set both flags.",
  "confidence":0.86,"past_pr_number":null}]

// chunk 2 — billing.ts (the one with memory)
[{"line":88,"severity":"suggestion",
  "comment":"This adds a manual retry loop around Stripe. Your team rejected this pattern in PR #214
in favour of the shared idempotency wrapper.",
  "confidence":0.78,"past_pr_number":214}]

// chunk 3 — billing.ts second hunk
[]

// chunk 4 — Button.tsx
[]
```

3.8 The final GitHub review

No `error` severity is present, so `event="COMMENT"` (`post_comments.py:29–30`).

REVIEW BODY

```
## 😈 PR Review Agent

Found **2** potential points of interest in this pull request.

### `api/src/routes/auth.ts`

⚠️ **Line 42** — Session cookie is set without httpOnly/sameSite; an XSS on any page
can steal the remember-me token. Set both flags.
  *Confidence: 86%*

### `api/src/services/billing.ts`

💡 **Line 88** — This adds a manual retry loop around Stripe. Your team rejected
this pattern in PR #214 in favour of the shared idempotency wrapper.
  > Referenced from PR #214
  *Confidence: 78%*
```

INLINE COMMENT — API/SRC/ROUTES/AUTH.TS, LINE 42 (RIGHT SIDE)

```
⚠️ **WARNING**: Session cookie is set without httpOnly/sameSite; an XSS on any page
can steal the remember-me token. Set both flags.
*Confidence: 86%*
```

INLINE COMMENT — API/SRC/SERVICES/BILLING.TS, LINE 88 (RIGHT SIDE)

```
💡 **SUGGESTION**: This adds a manual retry loop around Stripe. Your team rejected
this pattern in PR #214 in favour of the shared idempotency wrapper.

> Referenced from past PR #214
*Confidence: 78%*
```

WHAT GITHUB SHOWS

A review whose state is **Commented** (not "Changes requested", because no severity was `error`), with two inline threads anchored to `auth.ts:42` and `billing.ts:88`. The commit keeps its separate status check: **Powerful AI / PR Risk — pending — "Risk Score: 59/100 (Moderate Risk)"**.

3.9 Side effects after completion

Record	Change	Values for this run
`PRReview`	completed	`riskScore=59`, `commentsCount=2`, `filesReviewed=3`, `commentUrl` set, `completedAt` set
`ReviewComment` ×2	inserted	`auth.ts:42` warning 0.86; `billing.ts:88` suggestion 0.78 (`pastPrNumber=214`)
`UsageEvent`	inserted	`pr_review`, `llama-3.3-70b-versatile`, `llmCalls=4`, token counts, duration
`Organization`	incremented	`monthlyReviewCount` 12 → 13
`Repo`	updated	`customRulesCount=0`

SSE EVENTS THE DASHBOARD RECEIVES

```
fetch_pr        running   → completed   (files_count: 3)
fetch_config    running   → completed   (rules_count: 0)
chunk_changes   running   → completed   (4 chunks)
search_memory   running   → completed   (memory_hits: 1)
score_risk      running   → completed   (risk_score: 59)
llm_review      running   → completed   (comments_count: 2)
post_comments   running   → completed
notify_complete running   → completed
done            completed                (2 comments posted to GitHub)
```

SECTION 04

Local runbook

Two ways to boot the stack, the variables that gate startup, and the exact log lines that prove each stage ran.

4.1 Option A — Docker Compose (recommended)

`docker-compose.yml` provisions Postgres 16 with pgvector, Redis, the API, the agent, and the frontend.

```
cp .env.example .env
# edit .env — see the table in 4.3
docker compose up -d --build

curl http://localhost:3000/health    # {"status":"ok","service":"pr-review-agent-api"}
curl http://localhost:8000/health    # {"status":"ok"}
# dashboard: http://localhost:3001     agent docs: http://localhost:8000/docs

docker compose exec api npx prisma db push
```

THE LAST COMMAND MAY FAIL IN THE RUNTIME IMAGE

`prisma` is a **devDependency** (`api/package.json:39`) and the runtime stage installs with `--omit=dev` (`api/Dockerfile:27`), so the CLI is absent. Reliable fallback from the host (the compose file publishes 5432):

```
cd api
npm install
# PowerShell
$env:DATABASE_URL="postgresql://postgres:postgres@localhost:5432/pr_reviewer?schema=public"
npx prisma db push
```

4.2 Option B — Bare metal

```
# 1. Postgres with pgvector + Redis
docker run -d --name pr-pg -p 5432:5432 \
  -e POSTGRES_USER=postgres -e POSTGRES_PASSWORD=postgres -e POSTGRES_DB=pr_reviewer \
  pgvector/pgvector:pg16
docker exec -it pr-pg psql -U postgres -d pr_reviewer -c "CREATE EXTENSION IF NOT EXISTS vector;"
docker run -d --name pr-redis -p 6379:6379 redis:alpine

# 2. Node API — terminal 1
cd api
npm install
npx prisma generate
npx prisma db push
npm run dev                  # nodemon.json: tsx src/app.ts

# 3. Python agent — terminal 2
cd agent
python -m venv venv
venv\Scripts\activate        # Windows; source venv/bin/activate on Unix
pip install -r requirements.txt
uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload

# 4. Frontend — terminal 3
cd frontend
npm install
npm run dev                  # Next.js on port 3001
```

4.3 Environment variables

Variable	Used by	Enforced / default
`SESSION_SECRET`	API	Hard startup error if missing — `api/src/app.ts:27`
`INTERNAL_SERVICE_KEY`	API + agent	Hard startup error — `internalAuth.ts:8`, `config.py:17`; header `x-internal-secret`
`GROQ_API_KEY`	agent	Hard startup error — `config.py:19`
`DATABASE_URL`	API + agent	Hard startup error — `config.py:20`; Prisma `prisma.config.ts:12`
`REDIS_URL`	API	Default `redis://localhost:6379` — `api/src/lib/redis.ts:3`
`GITHUB_WEBHOOK_SECRET`	API	Required to verify payloads — `webhook.ts:9`
`GITHUB_APP_ID`, `GITHUB_PRIVATE_KEY`	API	Required to mint installation tokens — `octokit.ts:10,18`
`GITHUB_CLIENT_ID`, `GITHUB_CLIENT_SECRET`	API	Dashboard OAuth login — `README.md:240-241`
`AGENT_URL`	API	Default `http://localhost:8000` — `services/agent.ts:3`
`NODE_API_URL`	agent	Default `http://localhost:3000` — `config.py:9`

Variable	Used by	Enforced / default
`FRONTEND_URL`	API	Default `http://localhost:3001` — `app.ts:84`
`NEXT_PUBLIC_API_URL`	frontend	Browser-facing API base — compose line 68

Compose overrides `DATABASE_URL`, `REDIS_URL`, `AGENT_URL`, and `NODE_API_URL` to service hostnames (`docker-compose.yml:30-32, 49-52`).

4.4 Make a sample PR actually trigger a review

The review path needs a real GitHub App installation: `get_github_client()` raises `"Installation ID is required."` (`agent/app/services/github.py:27`) even though its docstring mentions a PAT fallback. Only onboarding uses `GITHUB_TOKEN` (`agent/app/graph/onboard/nodes/fetch_history.py:15`).

1. **Create a GitHub App** with callback `http://localhost:3000/auth/github/callback`, setup URL `http://localhost:3000/auth/github/installed`, and a webhook URL pointing at your tunnel.
2. **Grant permissions** — Pull requests: Read & write; Issues: Read & write; Contents: Read-only; Metadata: Read-only (`README.md:138-144`). Subscribe to **Pull request** events (`README.md:148`).
3. **Download the private key** into `GITHUB_PRIVATE_KEY`, keeping the `\n` escapes — `octokit.ts:10` converts them back.
4. **Start the tunnel** (README points at Smee/ngrok; `smee-client` is not a repo dependency):

   ```
   npx smee-client --url https://smee.io/<your-channel> \
     --target http://localhost:3000/webhooks/github
   ```

5. **Install the App** on a throwaway repo. Create a branch, edit three files (one under an `auth`/`session` path, one under `billing`/`payment`, one UI file), push, and open the PR.

4.5 Logs that prove each stage

API PROCESS

```
API server running on port 3000
Redis connected
→ Review job 1 queued for PR #482 on acme/shop (db id: clxr482)
[Pipeline] clxr482 — fetch_pr: running
[Pipeline] clxr482 — fetch_pr: completed
... one pair per node ...
✓ Review clxr482 marked as completed with 2 comments
```

WORKER (SAME API PROCESS)

```
Processing review job 1 for PR #482
✓ Review completed for PR #482 (7312 tokens in 41230ms)
```

AGENT

```
[Node 1] Fetching diff for PR #482 on acme/shop
[Node 1] Got 3 changed files (head_sha: 9f3c2ab)
[Node fetch_config] Checking for repository custom rules on acme/shop
[Node fetch_config] No .powerful.yml detected; using standard convention checks
[Node 2] Created 4 chunks
[Node 3] Searching memory for 4 chunks
[Node 3] Found memory context for 1/4 chunks
[Node score_risk] Assessed PR #482: 59/100 (Moderate Risk)
[Node 4] Reviewing 4 chunks with LLM
[Node 4] Generated 2 comments (4 LLM calls, 7312 tokens)
[Node 5] Posting 2 comments to GitHub
[Node 5] Posted review: https://github.com/acme/shop/pull/482#pullrequestreview-...
[Node 6] Review complete for PR #482
[Node 6] Notified Node API — job clxr482 complete
[Progress] fetch_pr:running → http://localhost:3000/internal/pipeline-update
[Progress] Response: 200
```

4.6 Verify the result

```
# Live pipeline (no auth on this route — app.ts:106)
curl -N http://localhost:3000/api/pipeline/stream/clxr482

# Persisted row
docker exec -it pr-pg psql -U postgres -d pr_reviewer \
  -c 'SELECT id, "prNumber", status, "riskScore", "commentsCount" FROM "PRReview";'
```

4.7 Faster loop — call the agent directly

Skips webhook and queue, but still needs a valid installation id and a real PR:

```
curl -X POST http://localhost:8000/review \
  -H "Content-Type: application/json" \
  -H "x-internal-secret: $INTERNAL_SERVICE_KEY" \
  -d '{"job_id":"manual-1","repo":"acme/shop","pr_number":482,
       "installation_id":12345678,"confidence_threshold":0.5,
       "past_dismissal_rate":0.2}'
```

It will post a real review to GitHub. No `PRReview` row exists, so `/internal/review-complete` is a no-op (`updateMany` , `internal.ts:35`) and comments are not persisted; SSE still works under `manual-1` .

4.8 Troubleshooting

Symptom	Likely cause	Fix
401 “Invalid signature”	`GITHUB_WEBHOOK_SECRET` differs from the App's secret	Copy the exact secret into `.env` ; restart the API
500 “Webhook secret not configured”	Variable missing	Set `GITHUB_WEBHOOK_SECRET`
403 “Unauthorized internal service request”	API and agent keys differ	Use one `INTERNAL_SERVICE_KEY` for both services

Symptom	Likely cause	Fix
No review, only an issue comment	`create_review` rejected an inline line, fallback fired	Expected on bad LLM line numbers; check agent logs for “falling back”
200 with `monthly_quota_exceeded`	Org hit its plan limit	Raise `plan` or reset `monthlyReviewCount`
200 with `duplicate_webhook`	Same event within 60 s	Wait a minute or change the head SHA
Agent exits at startup	Missing `INTERNAL_SERVICE_KEY`, `GROQ_API_KEY`, or `DATABASE_URL`	Set them; the validation message names the missing ones
Empty memory / no past decisions	Onboarding never ran	Trigger `POST /api/repos/:id/sync` (`repos.ts:34`)

SECTION 05

Interview gotchas

Things the code does that the README does not say — useful for showing depth, and for avoiding overclaims.

5.1 Seven details worth raising

1. **No PAT fallback in the review path.** The docstring at `agent/app/services/github.py:5–9` says it falls back to a personal token, but line 27 raises `"Installation ID is required."`. Only onboarding accepts `GITHUB_TOKEN`.
2. **Memory retrieval is repo-scoped, not org-scoped.** `search_memory()` filters only on `repoId` (`memory.py:90–92`), while `MemoryEntry` stores `orgId` and the README claims org+repo scoping.
3. **The embedding model in the README is wrong.** Code uses `BAAI/bge-small-en-v1.5` (`embeddings.py:4`); README says `all-MiniLM-L6-v2`. Both are 384-dimensional.
4. **Two commit statuses per review.** `score_risk.py:116` posts one, then `reviewWorker.ts:85` posts another with the same context; the worker's wins and adds a dashboard link.
5. **LLM line numbers are trusted.** `post_comments.py:51` passes them straight to GitHub. An invented line makes `create_review` throw, and the fallback collapses everything into one plain PR comment with no inline threads.
6. **Two writers update the same row.** `notify_complete` (`internal.ts:35`) and the worker (`reviewWorker.ts:60`) both write `PRReview`; values agree, but it is duplication.
7. **No tests, no CI, no LICENSE file** in the tree. Verification today is manual PRs plus log inspection; say that plainly rather than implying a test suite.

5.2 Where to say “I'm not sure”

- **Why `5c39865` is called “upgraded”.** The message says nothing; the content is inferred from `git show --stat`.
- **Whether the weekly digest is emailed.** `cron.ts:39` enqueues a job and `digestQueue.ts` exists, but I did not trace the worker to a mail provider.
- **The frontend SSE consumer.** The server endpoint and handler are clear (`app.ts:106`, `internal.ts:93`); the browser component I did not open.
- **Whether `docker compose exec api npx prisma db push` works as written.** The runtime image omits devDependencies, so verify before relying on it; the host-side fallback in 4.1 is safer.

> **THE 30-SECOND SUMMARY**
>
> “GitHub sends a signed webhook. The Node API verifies it, enforces org/repo/quota rules, writes a pending review row, and queues a BullMQ job. A worker enriches the job with the org's sensitivity and dismissal history, then calls a FastAPI service. A LangGraph pipeline fetches the diff, loads `.powerful.yml`, slices it into 30-line chunks, recalls similar past decisions from pgvector, scores risk, asks Groq's llama-3.3-70b for JSON findings per chunk, and posts a real GitHub review with inline comments. Every node streams progress through Redis to an SSE dashboard, and the result is persisted with token usage and a monthly counter.”

APPENDIX A

File map of the review path

Every file that participates in a PR review, in execution order.

File	Key symbol	Role
`api/src/app.ts`	`app.listen`, route mounting	Process entry; raw body for `/webhooks`; SSE route at line 106
`api/src/middleware/webhook.ts`	`verifyWebhookSignature()`	HMAC-SHA256 check
`api/src/routes/webhooks.ts`	`router.post('/github')`	Guards, org/repo/review rows, enqueue
`api/src/lib/cron.ts`	`checkAndResetOrgQuota()`	Quota self-heal; digest and monthly reset schedules
`api/src/queues/reviewQueue.ts`	`new Queue('review-queue')`	Retry policy and job data type
`api/src/workers/reviewWorker.ts`	`new Worker(...)`	Context enrichment, agent call, persistence, telemetry
`api/src/services/agent.ts`	`triggerReview()`	HTTP client to the agent
`agent/app/main.py`	`lifespan()`	Startup env validation
`agent/app/api/routes.py`	`review_pr()`, `verify_internal_secret()`	Agent ingress and graph invocation
`agent/app/graph/review/agent.py`	`build_review_graph()`	Node order and wiring
`agent/app/graph/review/state.py`	`ReviewState`, `ReviewComment`	Shared state schema
`fetch_pr.py` + `services/github.py`	`fetch_pr()`, `get_pr_details()`	Token exchange and diff fetch
`api/src/routes/internal.ts`	`/installation-token`, `/review-complete`, `/pipeline-update`, `handlePipelineStream()`	Machine-to-machine endpoints
`api/src/lib/octokit.ts`	`getInstallationOctokit()`, `createCommitStatus()`	JWT signing, token cache, status checks
`fetch_config.py`	`fetch_config()`	`.powerful.yml` rule loading
`chunk.py`	`chunk_changes()`	30-line slicing with line ranges
`search.py` + `services/memory.py`	`search_memory_for_chunks()`, `search_memory()`	pgvector cosine recall

File	Key symbol	Role
`services/embeddings.py`	`embed()`	384-dim bge-small embeddings
`score_risk.py`	`score_risk()`	Four-bucket risk score
`llm_review.py` + `services/llm.py`	`llm_review()`, `llm`	Prompting, JSON parsing, confidence filter
`post_comments.py`	`post_comments()`	Event choice and Markdown formatting
`services/github.py`	`post_pull_request_review()`, `post_commit_status()`	GitHub writes and fallback
`notify.py`	`notify_complete()`	Completion callback
`services/progress.py`	`report_progress()`	Fire progress events
`api/src/lib/pipelineEvents.ts`	`emitPipelineEvent()`	Redis pub/sub + history
`api/prisma/schema.prisma`	`Organization`, `Repo`, `PRReview`, `ReviewComment`, `MemoryEntry`, `FeedbackAction`, `UsageEvent`	Tenancy, reviews, vector memory, telemetry

APPENDIX B

Full commit history

All 19 commits, oldest first. The repository has no twentieth commit, so “first 30” is the whole history.

Hash	Date	Subject	Phase
`4beb79f`	2026-07-19	Initial commit - AI PR Reviewer	Initial
`afc4c95`	2026-07-19	Fix frontend submodule folder	Fix
`e51d303`	2026-07-19	fix: clean up review route, remove header type errors	Fix
`eb69b8e`	2026-07-19	fix: set module and moduleResolution to Node16	Fix
`07f4b75`	2026-07-19	new	Fix
`800e280`	2026-07-20	fix: resolve typescript compilation errors for render deployment	Fix
`01a36b9`	2026-07-20	fix: allow cross-site cookies for vercel frontend	Fix
`cac6b7d`	2026-07-20	fix: look up org login from GitHub API in installation callback	Fix
`27d4457`	2026-09-12	fixes all inconsistency	Redesign
`5116b5b`	2026-09-12	redesign frontend	Redesign
`17c7358`	2026-09-12	latest	Redesign
`5c39865`	2026-09-19	upgraded	Agent
`1d3bc06`	2026-09-19	hompage design	Homepage
`478dd74`	2026-09-19	hompage interactive	Homepage
`5a43016`	2026-09-19	hompage design part 2	Homepage
`6e346e8`	2026-09-20	version upgraded	Homepage
`b5f5ab2`	2026-09-20	fix	Homepage
`fe9b4e6`	2026-09-20	fix hero page	Homepage
`9d80155`	2026-09-20	again fix	Homepage

Generated from the repository at `D:\users\AI-ML\powerful` on 25 October 2026. Claims cite file paths, function names, and commit hashes as they exist in the working tree. Where the code and README disagree, both are shown. Constructed examples are labelled as such.